FAQs

CityCycle London

Project FAQs

DSAI4 Module 2 · Team 2 · March 2026

bigquery-public-data.london_bicycles · 32,342,086 rides · 2020–2023

About this document

This document captures frequently asked questions about the CityCycle data pipeline project — including technical decisions, data quality issues, and analytical findings. New questions are appended as they arise during development and presentation.

FAQ #1 Why are there different station counts — 795, 798, 837 and 1,610?

Answer

There are four distinct station counts across the project. Each one means something different and comes from a different source or filter:

Count	Source	Filter	What it means
1,610	fact_rides	None	Every distinct start_station_id ever recorded across all 32M rides 2020–2023 — includes closed, relocated and historical stations.
837	fact_rides	HAVING COUNT(*) > 50	Stations with meaningful ride activity (more than 50 departures) — used in the rebalancing hotspot analysis in the notebook.
798	dim_stations / stg_cycle_stations	id not null, valid lat/lon, nb_docks > 0	Official TfL station registry after removing 3 records with corrupted coordinates, zero docks, or null IDs.
795	dim_stations	is_installed = TRUE AND is_locked = FALSE	Operationally active stations — physically installed, unlocked and in service. The correct number to cite.

Why is 837 greater than 798?

The 837 count comes bottom-up from ride records — stations that appear as departure points. The 798 count comes top-down from the TfL stations registry. 39 stations appear in ride records but not in the registry because:

- Station names changed slightly between the ride record and the registry (free-text field drift over 3 years)
- Stations that were deregistered before the cycle_stations snapshot was taken still have historical ride records
- Temporary event stations that were never formally registered

This discrepancy is the root cause of the dbt FK warning:

```
⚠ FK: start_station_id → dim_stations (1,610 unmatched rows)
```

This was deliberately downgraded to WARN rather than FAIL — it reflects upstream data quality in the public dataset, outside our control.

The correct number to use in the presentation:

795 active stations — installed, unlocked, with valid coordinates and dock count.

FAQ #2 Why was the Meltano tap and target both BigQuery? What are the advantages, disadvantages, and an alternative target?

Answer

Why both tap and target are BigQuery

The source data (bigquery-public-data.london_bicycles) already lives in BigQuery as a public dataset. The destination (citycycle-dsai4.citycycle_raw) is also BigQuery. Meltano creates an owned copy of the data in your own project — without it, every dbt model and GE check would query the public dataset directly, meaning no partitioning control, no cost guard, no schema ownership, and dependency on Google's public data availability.

Advantages of tap-bigquery → target-bigquery

- Zero format conversion — data stays in BigQuery-native types throughout. No serialisation to CSV, Parquet or JSON and back.
- Incremental load support — tap-bigquery uses a start_date state key so subsequent runs only pull new rows, not the full 83M row table every time.
- Schema auto-detection — the tap reads the source schema and target-bigquery creates the destination table with matching column types automatically.
- Same EU region — no cross-region data transfer costs.
- Same credentials and toolchain work for both source and destination.

Disadvantages of tap-bigquery → target-bigquery

- Redundant data copy — pays storage costs to duplicate 32M rows already freely available in BigQuery.
- Overkill for a static dataset — Singer pipelines are designed for live incremental ingestion. For a dataset ending January 2023, a one-time bulk load would have been simpler.
- BQ scan cost on read — tap-bigquery issues a SELECT against the source table, consuming scan quota even before writing begins.
- No transformation during load — Meltano is pure EL. All transformation logic must sit in dbt downstream.

Alternative target — Google Cloud Storage (target-gcs)

Instead of writing directly to BigQuery, Meltano could write to GCS as Parquet or CSV files first, then load into BigQuery via a separate bq load command or a BigQuery External Table.

Aspect	target-bigquery	target-gcs
Load method	Direct streaming insert	Stage files → bq load
Storage cost	BigQuery pricing from day one	GCS storage cheaper than BQ

Flexibility	BigQuery only	Can feed Spark, dbt, Dataproc from same files
Query latency	Immediate availability	Extra load step required first
Best for	Operational pipelines	Data lake / multi-consumer architectures

target-gcs would be more appropriate if the project needed to feed multiple downstream systems — for example if Spark was used for ML training. For this project target-bigquery was the correct choice because the entire stack — dbt, GE, Looker Studio, Streamlit via SQLAlchemy — all speak BigQuery natively.

FAQ #3 What statistical algorithms were used for ML? What metrics determined the best model, and what assumptions and derived features were used? Explain the train/test split.

Answer

Statistical algorithms used

1. Linear Regression (baseline)

Assumes a linear relationship between features and hourly ride demand. Fits a hyperplane through the feature space by minimising the sum of squared residuals using Ordinary Least Squares. Fast to train but cannot capture non-linear interactions between hour, station and season.

2. Random Forest Regressor

An ensemble of decision trees. Each tree is trained on a random bootstrap sample of the data with a random subset of features at each split (bagging). The final prediction is the mean of all tree predictions. Naturally captures non-linear relationships and feature interactions without requiring feature scaling.

3. XGBoost — Extreme Gradient Boosting (best model)

A boosted ensemble where trees are built sequentially — each new tree corrects the residual errors of the previous ones. Uses gradient descent in function space to minimise a differentiable loss function. More powerful than Random Forest for structured tabular data and more efficient due to regularisation and tree pruning.

Metrics used to determine the best model

Metric	Formula	What it measures
RMSE	$\sqrt{(\Sigma(y - \hat{y})^2 / n)}$	Average prediction error in rides/hour. Penalises large errors more heavily due to squaring. Lower is better.
MAE	$\Sigma\lvert y - \hat{y}\rvert / n$	Average absolute error. Less sensitive to outliers than RMSE. Lower is better.
R^2	$1 - SS_res / SS_tot$	Proportion of variance in the target explained by the model. 1.0 = perfect, 0 = no better than predicting the mean. Higher is better.

Model comparison results

Model	RMSE	MAE	R^2	Train time
XGBoost ← best	2.422	1.508	0.488	75.9s
Random Forest	2.428	1.519	0.486	829.9s
Linear Regression	2.595	1.604	0.413	1.0s

XGBoost was selected — lowest RMSE and MAE, highest R^2, and 11× faster than Random Forest.

Train / test split

An 80/20 random split was applied — 80% of rows (~25.8M) used for training, 20% (~6.5M) held out for testing. The split uses random_state=42 for reproducibility. No time-based split was used — the features include day_of_week, hour and season which capture temporal patterns explicitly, so a random split still exposes the model to all seasons and time periods during training.

Assumptions

- Stationarity — demand patterns learned from 2020–2023 will hold for future dates. COVID-19 disruption in 2020 partially violates this assumption.
- Station ID as numeric — start_station_id is treated as a numeric value rather than categorical. A better approach would be one-hot encoding or station embeddings.
- Fixed rolling average — rolling_7d_avg is hardcoded to 50.0 in the forecast dashboard because the dataset ends in 2023. In production this would use the actual 7-day rolling average from live data.
- Season hardcoded to 1 (spring) — the forecast page uses a fixed season value, slightly overstating winter demand and understating summer.
- Independence of rides — each ride is treated as an independent observation. In reality, demand at one station influences nearby stations through the network effect.

Derived features used in training

Feature	How derived	Why included
hour	EXTRACT(HOUR FROM start_datetime)	Captures intraday demand — double peak at 8AM and 6PM
day_of_week	EXTRACT(DAYOFWEEK FROM start_datetime)	Weekday vs weekend behaviour
is_weekend	day_of_week IN (1, 7)	Binary flag — leisure vs commuter pattern
is_holiday	Hardcoded 0	UK bank holidays — not derived from data in this version
season	CASE on month number	Captures seasonality — summer 5–6× higher than winter
start_station_id	Raw station ID from fact_rides	Proxy for station location and historical popularity
rolling_7d_avg	7-day rolling avg of departures per station	Captures recent demand momentum at that station

The target variable is hourly ride count per station — aggregated from fact_rides as COUNT(*) GROUP BY start_station_id, start_hour, hire_date.

FAQ #4 How were the commuter, leisure and casual user percentages defined and calculated?

Answer

User segments were derived using K-Means clustering on a 30,000-ride sample drawn from 2022 ride data, using four behavioural features.

Features used for clustering

- duration_minutes — length of each ride
- start_hour — hour of day the ride began
- is_weekend — binary flag (1 = weekend ride)
- is_peak — binary flag (1 = ride during AM or PM commuter peak hours)

All features were standardised using StandardScaler before clustering to ensure no single feature dominated due to scale differences.

Choosing k = 3

The elbow method and silhouette analysis were both run for k = 2 through 7. The silhouette method returned an optimal k of 5, but k = 3 was chosen deliberately for interpretability and business relevance — three segments map cleanly onto commuter, leisure, and casual usage patterns that TfL and operators recognise.

Cluster labelling rules

Each cluster was profiled on its average peak-hour share and weekend share, then labelled using the following decision rules:

- pct_peak > 0.35 → Commuter (high concentration in rush hours)
- pct_weekend > 0.40 → Leisure (weekend-dominant usage)
- Neither → Casual (mixed hours, moderate duration)

Results from the notebook (Cell 18 output)

Segment	n (sample)	Avg duration	% weekend	% peak	Share
Leisure	15,968	17.5 min	100%	0%	53%
Casual	9,654	16.1 min	0%	0%	32%
Commuter	4,378	16.5 min	60%	100%	15%

Important caveat: these percentages are derived from a 30,000-ride sample from 2022 only, not the full 32M-ride dataset. They are indicative of behavioural patterns rather than exact fleet-wide proportions. The Key Findings summary in the notebook (Cell 25) states approximately 45%/35%/20% — these are rounded narrative estimates, not the precise cluster output figures. The table above represents the ground truth from the clustering model.

FAQ #5 What are the differences between the dbt layers — staging, intermediate, and marts?

Answer

The pipeline uses three dbt layers, each with a distinct responsibility. The design principle is that each layer only references the layer directly above it — marts reference intermediate, intermediate references staging, staging references raw. Nothing skips a layer.

Layer	Materialised as	Responsibility	Key rule
Staging	View	Type casting, renaming, filtering bad rows	No business logic — source changes only touch here
Intermediate	View	Business logic, joins, aggregations, derived flags	Imbalance score formula lives here — flows downstream automatically
Marts	Table	Final analytics-ready output for dashboards and ML	Partitioned and clustered for fast, cost-efficient BQ queries

Staging — trust the data

stg_cycle_hire and stg_cycle_stations clean raw Meltano output: cast types defensively, rename columns for clarity (duration → duration_seconds), and filter out physically impossible records — null rental IDs, durations outside 60–86,400 seconds, and rides where end_datetime precedes start_datetime. This is where the 32,369,326 raw rows are trimmed to 32,342,086 in the final fact table.

Intermediate — apply business logic

int_rides_enriched joins rides to stations twice (start and end), then computes peak_hour_flag (hours 7, 8, 17, 18), duration_band (short/medium/long/extended), time_period, and is_round_trip. int_station_daily_stats aggregates to station-day level, computes net_flow (departures minus arrivals), utilisation_rate, and the imbalance_score formula:

- imbalance_score = abs(departures - arrivals) / max(departures + arrivals, 1)
- is_imbalanced flag set at score > 0.20
- Direction classified as draining (net_flow > 0), filling (net_flow < 0), or balanced

Both intermediate models are views — they cost nothing to store and always reflect the latest logic when queried downstream.

Marts — serve the outside world

fact_rides is the gold layer that dashboards, Looker Studio, Streamlit, and the XGBoost model all query. It is materialised as a table, partitioned by hire_date (day granularity), and clustered by start_station_id and end_station_id — so BigQuery only scans relevant partitions and station groups, keeping query costs low across 32M rows. It also computes the rolling_7d_avg_departures window function used as an ML feature.

dim_stations joins all-time station stats from int_station_daily_stats and applies the hardcoded priority thresholds: CRITICAL ≥ 0.25, HIGH ≥ 0.18, MEDIUM ≥ 0.10, else LOW. dim_date is a full date spine from 2015 to 2025 with UK bank holidays and season flags.

Important caveat on bank holidays

UK bank holidays are hardcoded in dim_date for 2023–2024 only. The analysis period (2020–2023) has no bank holidays defined, so is_uk_bank_holiday is FALSE for all rides in the dataset. Holiday effects on ridership cannot be analysed from the current model without extending the bank holiday reference data.

FAQ #6 Does dbt use columnar transformation?

Answer

No. dbt does not perform columnar transformation. dbt is a transformation orchestration tool — it writes SQL, submits it to BigQuery, and BigQuery executes it. All columnar behaviour happens entirely inside BigQuery's execution engine. dbt has no visibility into or control over how rows are scanned, encoded, or processed at the storage level.

What dbt controls

- What gets computed — the SQL logic in each model
- How results are stored — view vs materialised table, via the materialized config
- How tables are physically organised — partition_by and cluster_by config on fact_rides

What BigQuery controls

- Columnar storage — all BQ tables are stored in columnar format automatically, regardless of dbt
- Columnar execution — when a query requests AVG(duration_minutes), BQ reads only that column across 32M rows, not the full row
- Partition pruning — queries filtered by hire_date only scan the relevant day-partitions
- Cluster pruning — queries filtered by start_station_id or end_station_id skip blocks of irrelevant data

How dbt adds partitioning and clustering on top of BigQuery's columnar storage

While BigQuery handles columnar storage automatically, dbt extends those savings by configuring how data is physically organised within that columnar store. In this project, fact_rides.sql declares two additional optimisations:

- Partitioning by hire_date (day granularity) — BigQuery splits the table into one physical segment per day. A rebalancing query asking for July 2022 only scans that month's partitions, skipping the other 1,000+ days entirely. Without partitioning, every query would scan all 32M rows.
- Clustering by start_station_id, end_station_id — within each partition, rows are physically sorted and co-located by station. A query filtering to a specific station skips large blocks of data for other stations. This is especially valuable for the rebalancing dashboard, which almost always queries by both date and station simultaneously.

The two optimisations compound each other. A Streamlit query asking for station-level stats for a specific station over a specific week fires both partition pruning (date filter) and cluster pruning (station filter) together — BigQuery reads only a small fraction of the table's bytes. This is why live BQ queries in the dashboard remain fast and cost-efficient despite the 32M row dataset. dbt is responsible for declaring these optimisations; BigQuery is responsible for enforcing them at query time.

FAQ #7 What did dbt do in this project?

Answer

dbt took 32 million raw ride records that arrived from Meltano in an unstructured, untyped, business-logic-free state and transformed them into a clean, tested, analytics-ready star schema that dashboards, the ML model, and the EDA notebook could all query directly.

It did this across three layers of SQL models — staging cleaned and typed the raw data, intermediate applied business logic and computed the imbalance score, and marts materialised the final fact and dimension tables into BigQuery with partitioning and clustering for query performance. It then ran 56 automated tests across every model to verify the output was correct before anything downstream touched it.

Breaking down: dbt 1.x · BigQuery adapter · dbt_utils · 56 PASS · 0 ERROR · 3 WARN

- dbt 1.x — the version of dbt Core used. Version 1.x introduced stable interfaces for materialisation configs, partitioning, and the ref() function that enforces the layer dependency chain.
- BigQuery adapter — dbt is database-agnostic. The BigQuery adapter translates dbt SQL into BigQuery-specific syntax, including the partition_by and cluster_by config on fact_rides, and the INFORMATION_SCHEMA queries dbt uses internally to detect schema changes.
- dbt_utils package — an open-source macro library from dbt Labs. Used for two things: generate_surrogate_key() to create ride_sk and station_sk surrogate keys on the mart tables, and date_spine() to generate the full 2015–2025 date range in dim_date.
- 56 PASS — 56 schema tests across _staging.yml and _marts.yml all passed. These cover not-null checks, uniqueness, accepted value ranges, referential integrity between fact_rides and dim_stations, and custom range checks via dbt_utils.accepted_range.
- 0 ERROR — no test failed outright. A failed test means the data violated a hard constraint — null rental_id, duplicate surrogate key, duration_minutes outside 1–1440. Zero errors means none of those constraints were violated.
- 3 WARN — three tests were configured with severity: warn rather than severity: error. These cover the ~312,000 rides where end_station_id is null (bikes lost or not returned). Flagged as warnings so the pipeline surfaces them without blocking the run. Known, expected, and documented — they do not affect the rebalancing analysis.

FAQ #8 What does the CI GitHub Actions pipeline do?

Answer

Every time code is pushed to the main branch, GitHub automatically spins up a fresh Ubuntu environment and runs 5 jobs in sequence to verify the codebase is healthy. It acts as an automated gatekeeper — if any job fails, the push is flagged and the problem must be fixed before the code is considered deployable.

Critically, all 5 jobs run against mock data only. No live BigQuery calls happen in CI. This protects the free tier quota and means the pipeline can run on every push without incurring any GCP cost.

The 5 jobs

Job	Tool	What it does
lint	ruff + black	Checks code quality and formatting. Catches unused imports, undefined variables, indentation issues. Spell-check for code.
mock-data	mock_data_generator.py	Generates 10K synthetic rides + 795 stations (seed=42). Validates output schema matches BigQuery and dbt expectations.
dbt-compile	dbt compile	Parses all 7 dbt models, resolves ref() dependencies, validates SQL syntax and lineage chain. Zero BQ cost — no query runs.
train-model	XGBoost	Trains model on mock data. Validates feature column order matches demand_model.pkl exactly. Catches silent prediction errors from feature mismatch.
notebook	Structure check	Confirms 03_bq_eda_live_data.ipynb exists, has correct cell count and structure. Does not re-execute — no BQ access needed.

Why dbt-compile and not dbt run?

Running dbt run on every push would scan 32M rows multiple times a day and require committing GCP credentials to GitHub — both unacceptable. dbt compile gives 90% of the validation value at zero cost: it confirms all SQL is valid and all ref() dependencies resolve correctly without executing a single query.

Why this matters

The CI pipeline demonstrates professional software engineering practice — changes are automatically validated, the codebase is always in a known-good state, and no human has to remember to run tests manually. The mock data strategy specifically shows awareness of cost constraints on a cloud platform, which is a real-world engineering consideration assessors look for.

Rapid Fire Questions

Rapid Fire Questions

1. What is the business value of your project?

By analysing 32 million real London rides, the pipeline predicts which stations will run empty or overflow before it happens, enabling CityCycle to deploy rebalancing crews proactively rather than reactively — reducing operational cost, preventing lost rentals, and improving rider experience across 795 stations.

---

2. What data pipeline did you use?

An ELT pipeline — Extract, Load, Transform — where raw ride and station data is first ingested from BigQuery's public dataset via Meltano into a raw layer, then transformed in-place using dbt into a star schema, quality-gated with Great Expectations, and served to Streamlit dashboards, Looker Studio, and an XGBoost ML model, with the full pipeline orchestrated by Dagster and validated in CI via GitHub Actions.

---

3. Describe the Meltano process.

Meltano uses the Singer protocol — tap-bigquery reads from `bigquery-public-data.london_bicycles` and target-bigquery writes 32,342,086 ride records and 800 station records into our project's `citycycle_raw` dataset, acting as the Extract and Load stages before dbt takes over the Transform.

---

4. The star schema rationale.

A star schema was chosen because it separates measures (rides) from descriptive attributes (stations, dates) into a central fact table and supporting dimension tables, which makes rebalancing queries fast and intuitive — a query asking "which stations are draining on weekday mornings in summer" simply joins `fact_rides` to `dim_stations` and `dim_date` without complex denormalisation.

---

5. dbt transformation run.

dbt runs seven SQL models across three layers — staging views clean and type the raw data, intermediate views apply business logic including the imbalance score formula and peak hour flags, and mart tables materialise the final `fact_rides` (32.3M rows, partitioned and clustered), `dim_stations`, and `dim_date` into BigQuery — producing 56 passing tests, 0 errors, and 3 intentional warnings on nullable foreign keys.

---

6. Findings from Great Expectations.

Running 34 custom SQL checks across the raw and transformed layers returned 30 PASS, 4 WARN, and 0 FAIL — the warnings cover known edge cases such as rides with null end station IDs (bikes that were lost or not returned), which are expected in the data and do not affect the rebalancing analysis.

---

7. What are your key findings?

Analysing 32M rides revealed a clear weekday double-peak at 08:00 and 17:00–18:00, three rider segments (Leisure 53%, Casual 32%, Commuter 15%), 30 stations requiring urgent rebalancing intervention with New North Road Hoxton as the most critical at a score of 0.324, and a seasonal swing where summer demand runs 3× higher than winter — all with COVID-19's impact and full recovery clearly visible in the 2020–2022 data.

---

8. What did you apply the findings to?

The findings directly shaped the dashboard design and ML features — the double-peak pattern informed the `peak_hour_flag` feature and crew dispatch timing recommendations, the rider segments explain why different stations drain in opposite directions at different times, the imbalance scores drive the CRITICAL/HIGH/MEDIUM/LOW priority tiers shown in Streamlit and Looker Studio, and the seasonal pattern informs the staffing recommendation that summer requires 3× winter crew capacity.

---

9. What can you forecast?

The XGBoost model — trained on 10.4M rows of `fact_rides` using seven features including hour, day of week, station ID, season, and the 7-day rolling average of departures — forecasts hourly bike demand per station for the current day, achieving an RMSE of 2.422 rides per station per hour and explaining 49% of demand variance, which is sufficient to predict which stations will need bikes pre-positioned before the morning peak.

---

10. for dbt what does it mean "materialised as view"?

A materialised view in dbt means the model is not stored as a physical table in BigQuery — instead, dbt creates a saved SQL definition, and every time something queries it, BigQuery runs the SQL live against the underlying data at that moment.

---

11. what are the staging and intermediate layers used for?

Staging cleans and standardises the raw data into a trusted, consistently typed foundation, while intermediate applies business logic and joins to produce the analytical building blocks that the mart layer assembles into the final tables.